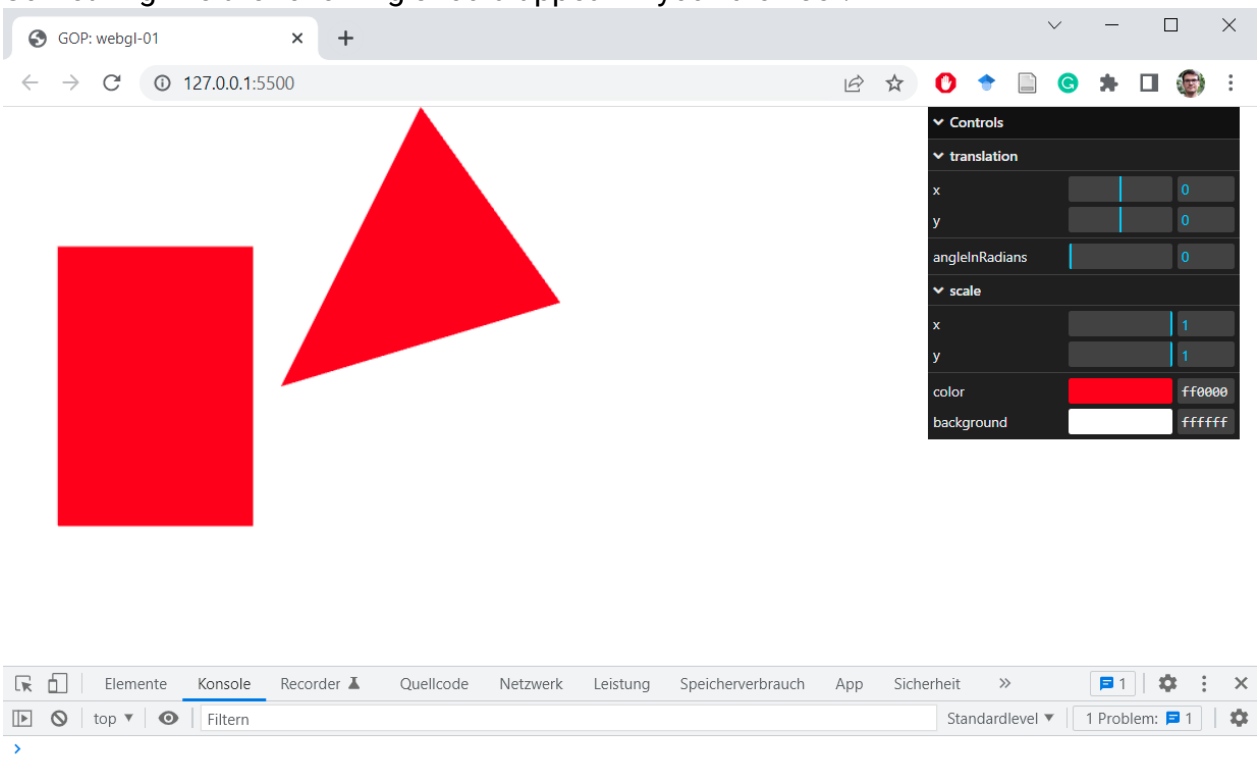
RTR-Lab: WebGL-1

1. Setup:

Accept the assignment in GitHub Classroom and clone the repository.
Open the repository in your favorite IDE and start a Server to show the index.html website in your browser:

- VSCode: You need the live server plugin. Then click “Go Live” to start your project.
- WebStorm: Right-click on “index.html” and “Run …” to start the internal webserver.

Something like the following should appear in your browser:



Use your browser's Developer Tools to debug and find errors. In Chrome and Firefox, you can open them with the Shortcut Ctrl+Shift+I or with F12. Keep them open, as errors will show up there!

2. Familiarize:

Get familiar with the WebGL code that is already running.
Look at and understand

- the shaders (`vs` and `fs` variables),
- the `*Buffer` functions (we discussed buffers/attributes in the lecture),
- the `*Shader` function in the `ShaderHelper.js` file,
- the `main` function, and
- the `drawScene` function.

The math classes for Vectors, Matrices, Quaternions, and colors are directly taken from the three.js framework, which has a neat API documentation (e.g., see Color).

The website draws a triangle and a quad (without index buffers) with a simple shader that transforms vertices with a `modelMatrix` and outputs the `color` from a uniform variable. Ignore the GUI code and any additional files for now!

3. Index Buffers:

Let's switch to index buffers for the quad. The buffers are already created but not used:

```
    // Indexed Geometry - Quad
    const quadIndexed = createBuffer(gl, [
        -0.1, -0.5, 0,
        -0.1, 0.5, 0,
        -.8, -0.5, 0,
        -.8, 0.5, 0
    ]);
    // Indexed Geometry - Quad
    const indices = createIndex(gl, [
            0, 1, 2,   // first triangle
            2, 1, 3,   // second triangle
    ]);
```

For rendering in the `drawScene` function, we need to bind the index buffer and switch from `drawArrays` to `drawElements`:

```
        // quad - bind buffer and set attribute pointers (CHANGED!)
        useBuffer(gl, quadIndexed, 3, positionLocation);
        // quad - bind the index buffer (NEW!)
        gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indices);
        // quad - draw the indexed geometry (NEW!)
        gl.drawElements(gl.TRIANGLES, 6, gl.UNSIGNED_SHORT, 0);
```

4. Color Attributes:

Next, we want to use a color attribute for our triangle and quad.
First, we need to prepare our shaders for using an additional attribute. Note the position was our only attribute so far!
We define the color as `in` variable in the vertex shader. Furthermore, we define an `out` variable for the color to pass it through the rasterizer:

```
var vs = `#version 300 es
layout(location=0) in vec3 position;
layout(location=1) in vec4 color; // RGBA (NEW)
uniform mat4 modelMatrix;

out vec4 v_color; // output to fs and interpolated by the rasterizer (NEW!)

void main() {
  gl_Position = modelMatrix * vec4( position, 1.0 );
  v_color = color; // (NEW)
}
`;
```

Thus, the fragment shader uses the color from the `in` variable for coloring the fragments:

```
var fs = `#version 300 es
precision highp float;

uniform vec3 color; // color input
out vec4 fragColor; // fragment color output
```

```
in vec4 v_color; // input from vertex shader (NEW)

void main() {
    fragColor = vec4( v_color.rgb, 1.0 ); // only assign the fragment color (NEW)
}
`;
```

If you look at our application (in the browser) the geometries are black. Which makes sense, since we did not define any colors yet!
Define a color buffer for the triangle and the quad. The colors are up to you.

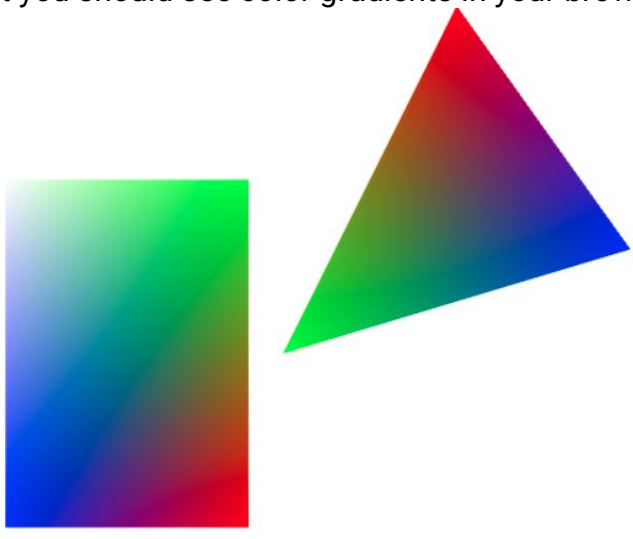
```
    // triangle color rgba
    const triangleColors = createBuffer(gl, [
        1, 0, 0, 1.0,
        0, 1, 0, 1.0,
        0, 0, 1, 1.0,
    ]);
    // quad color rgba
    const quadColors = createBuffer(gl, [
        1, 0, 0, 1.0,
        0, 1, 0, 1.0,
        0, 0, 1, 1.0,
        1, 1, 1, 1.0,
    ]);
```

Then use the buffers before drawing the triangle and the quad:

```
        // triangle with color
        useBuffer(gl, triangle, 3, 0);
        useBuffer(gl, triangleColors, 4, 1); // (NEW)
        gl.drawArrays(gl.TRIANGLES, 0, 3);

        // quad with color
        useBuffer(gl, quadIndexed, 3, 0);
        useBuffer(gl, quadColors, 4, 1); // (NEW)
        gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indices);
        gl.drawElements(gl.TRIANGLES, 6, gl.UNSIGNED_SHORT, 0);
```

If everything works out you should see color gradients in your browser now:



5. Cleanup/OOP BufferGeometry:

Finally, let's make our code cleaner by using OOP concepts.
Instead of all the `createBuffer/Index, useBuffer, gl.bindBuffer,` and `gl.draw*` commands we will use the class `BufferGeometry` (in the folder `./rtr/core/`).
It provides a set of functions for handling attributes (`setAttribute`) and indices (`setIndex`) and for drawing the geometries. Note that the `BufferGeometry` class uses lazy initialization, which means that the WebGL buffers are only initialized when used for the first time. Therefore, the constructor does not need a WebGL context (the `gl` variable), but the `draw` method does!
For the triangle, we can use the `BufferGeometry` like so:

```
// Triangle Geometry (NEW!)
const triangleGeometry = new BufferGeometry();
triangleGeometry.setAttribute("position", {
    'array':
        [
            .5, 1, 0,
            0, 0, 0,
            1, 0.3, 0,
        ],
    'itemSize': 3
});
triangleGeometry.setAttribute("color", {
    'array':
        [
            1, 0, 0, 1.0,
            0, 1, 0, 1.0,
            0, 0, 1, 1.0,
        ],
    'itemSize': 4
});
```

The quad with indices:

```
// Indexed Geometry - Quad (NEW!)
const quadIndexedGeometry = new BufferGeometry();
quadIndexedGeometry.setAttribute("position", {
    'array':
        [-0.1, -0.5, 0,
        -0.1, 0.5, 0,
        -.8, -0.5, 0,
        -.8, 0.5, 0
        ],
    'itemSize': 3
});
quadIndexedGeometry.setAttribute("color", {
    'array':
        [
            1, 0, 0, 1.0,
            0, 1, 0, 1.0,
            0, 0, 1, 1.0,
            1, 1, 1, 1.0,
```

```
            ],
        'itemSize': 4
    });
    quadIndexedGeometry.setIndex(
        [
            0, 1, 2,   // first triangle
            2, 1, 3,   // second triangle
        ]
    );
```

Drawing the triangle and quad then simplifies to a single line for each:

```
        triangleGeometry.draw(gl); //( NEW!)
        quadIndexedGeometry.draw(gl); //( NEW!)
```

The result in your browser should not change!

6. Cleanup/OOP WebGLProgram:

Furthermore, let's also use OOP concepts for our shaders and the shader program. Instead of all the `gl.useProgram, gl.uniform*, …` commands we will use the class `WebGLProgram`.

The WebGLProgram class is ready to use. We can create shaders with:

```
    // setup GLSL program
    const program = new WebGLProgram(gl); //( NEW!)
```

Note, that the shaders are hardcoded in the WebGLProgram. Any changes you want to apply to shaders needs to be done there, from now on.

Using shaders and setting uniforms simplifies to:

```
        program.use(); //( NEW!)
        program.setUniform('modelMatrix', matrix); //( NEW!)
        program.setUniform('color', controls.color ); //( NEW!)
```

Note that common datatypes such as Matrices, Vectors, or Colors are supported in the `setUniform` method!

Again, the result in your browser should not change, but your code should be cleaner.

7. Exercise:

Once you have completed and understood the steps above, you should be good to go. Now, start solving the tasks of the RTR exercise (the code that you need to hand in).

Have fun! 😊